

Codex CLI 架构解析

第一章 · 总览

一个以 **core** 为中心的本地 AI 代理操作系统 —— 理解它要解决的核心工程问题、与 Codex Web 的差异、分层架构如何分工，以及一条用户请求从输入到工具执行再返回模型的完整路径。

| 目录

01	执行摘要	3
02	项目定位与核心问题	4
03	系统边界与分层架构	6
04	核心数据流与关键抽象	8
05	扩展、安全与设计取舍	10
06	附录：关键实现入口	12

| 执行摘要

Codex CLI 是 OpenAI 开源的本地 AI 编程助手。本章回答四个问题：它要解决的核心工程问题是什么、与 ChatGPT / Codex Web 的差异在哪、它的分层架构如何分工、一条用户请求从输入到工具执行再返回模型的完整路径是什么。读懂本章，就等于拿到了后续所有章节的地图。

核心 Takeaways

- Codex CLI 是运行在用户本机的命令行智能体，关键设计约束是必须在本机安全地执行不可信代码(模型生成的命令)，同时提供流畅的交互体验。
- 架构以核心编排层 codex-core 为中心，把模型交互与本地执行解耦：tui 与 app-server 负责交互形态，exec-server 负责受控执行，core 只负责把输入编织成工具调用循环。
- 能力通过 Extensions、MCP、Skills、Plugins、Hooks 五种机制接入，大多数新增能力不需要修改核心循环。
- 安全是纵深防御加用户可控：分级权限 profile、沙箱模式、审批策略与 exec-server 进程隔离共同兜底。
- 代码库是单一大仓库 monorepo，Rust 工作区含 130+ crates(2026-08 为 134 个)，鼓励拆 crate 以避免 core 变成无法维护的巨型 crate。

本文档回答的问题

它要解决什么核心工程问题？它与 ChatGPT / Codex Web 的差异是什么？它的分层架构如何分工、一条请求如何从输入到工具执行再返回模型？

项目定位与核心问题

Codex CLI 的关键设计约束是必须在本机安全地执行不可信代码，同时提供流畅的交互体验——这两个目标直接塑造了它的安全执行层、交互层与核心编排层三层分工。

形态对比: CLI 是主体

形态	运行位置	主要交互	目标用户	与 CLI 的关系
Codex CLI(本仓库)	用户本地机器	终端 / TUI	喜欢命令行、需要本地安全的开发者	主体
Codex IDE 插件	IDE 内部	编辑器侧边栏	IDE 用户	复用同一套后端能力
Codex Desktop App	本地桌面应用	图形界面	偏好 GUI 的用户	通过 codex app 启动，复用 CLI 后端
Codex Web	OpenAI 云端	浏览器	需要云端算力和环境的用户	完全不同的云架构

Codex CLI 的关键设计约束是：必须在本机安全地执行不可信代码(模型生成的命令)，同时提供流畅的交互体验。这两个目标直接塑造了它的架构。

核心问题：为什么需要这样一套架构？

如果把 Codex CLI 看作一个会写代码的聊天机器人，它的工程挑战包括：模型生成的命令可能破坏用户系统；终端 UI 要实时显示流式输出、文件变化、工具状态；要支持桌面应用、远程模式、MCP 服务器等多种形态；不同用户或项目需要不同模型、权限、提示词；新工具不断涌现；长会话上下文会超过模型窗口；多 Agent 协作可能失控。

问题	朴素的解决方式	Codex CLI 的选择
模型生成的命令可能破坏用户系统	全部交给用户确认	分级权限 profile + 沙箱 + 可选自动审批
终端 UI 要实时显示流式输出、文件变化、工具状态	UI 和业务逻辑混在一个 crate 里	tui 与 core 分离，通过协议事件通信
支持桌面应用、远程模式、MCP 服务器等多种形态	为每种形态复制一套逻辑	引入 app-server 作为统一协议层
不同用户/项目需要不同模型、权限、提示词	硬编码默认配置	分层配置系统(系统/用户/项目/CLI 覆盖)
新工具(搜索、图片生成、记忆)不断涌现	改核心循环加工具	Extension API + MCP + Skills 三种扩展机制
长会话上下文会超过模型窗口	简单截断历史	Compaction、token budget、context fragments 分层
多 Agent 协作可能失控	单 Agent 串行	AgentControl 注册表、预算、并发限制、层级通信

这些选择共同指向一个目标：把模型交互与本地执行解耦，把核心循环与具体能力解耦，从而让系统可以持续演进而不变成巨型 crate。

| 系统边界与分层架构

仓库根目录下有两个子系统：极薄的 `codex-cli` TypeScript 包装，与包含 130+ crates 的 `codex-rs` Rust 工作区 (2026-08 为 134 个)。后者按六层逻辑组织，是 Codex CLI 的实际实现。

两个子系统

`codex-cli` 是一个极薄的 TypeScript 包装，负责按平台找到 Rust 二进制并转发参数；`codex-rs` 是 Rust 工作区，包含 130+ crates (2026-08 为 134 个)，是实际实现。

分层架构

下面是 Rust 工作区的逻辑分层 (不是目录顺序)：

用户可见层 (CLI / TUI / Desktop / Remote)	
<code>codex-rs/cli</code> , <code>codex-rs/tui</code> , <code>codex-rs/app-server-*</code>	
应用协议层 (AppServer Protocol + 状态同步)	
<code>codex-rs/app-server-protocol</code> , <code>app-server-client</code>	
核心编排层 (会话、Thread、Turn、Agent 控制)	
<code>codex-rs/core</code> (以及被拆出的 <code>core-api</code> , <code>core-plugins</code>)	
能力扩展层 (Skills / MCP / Plugins / Extensions / Hooks)	
<code>codex-rs/skills</code> , <code>codex-rs/codex-mcp</code> , <code>codex-rs/hooks</code> , <code>ext/*</code>	
本地执行层 (沙箱、进程、文件系统、网络代理)	
<code>codex-rs/exec</code> , <code>codex-rs/exec-server</code> , <code>sandboxing</code> , <code>bwrap</code>	
基础设施层 (模型调用、配置、持久化、遥测、认证)	
<code>codex-rs/backend-client</code> , <code>config</code> , <code>state</code> , <code>rollout</code> , <code>history</code> ,	
<code>codex-rs/login</code> , <code>otel</code>	

各层职责

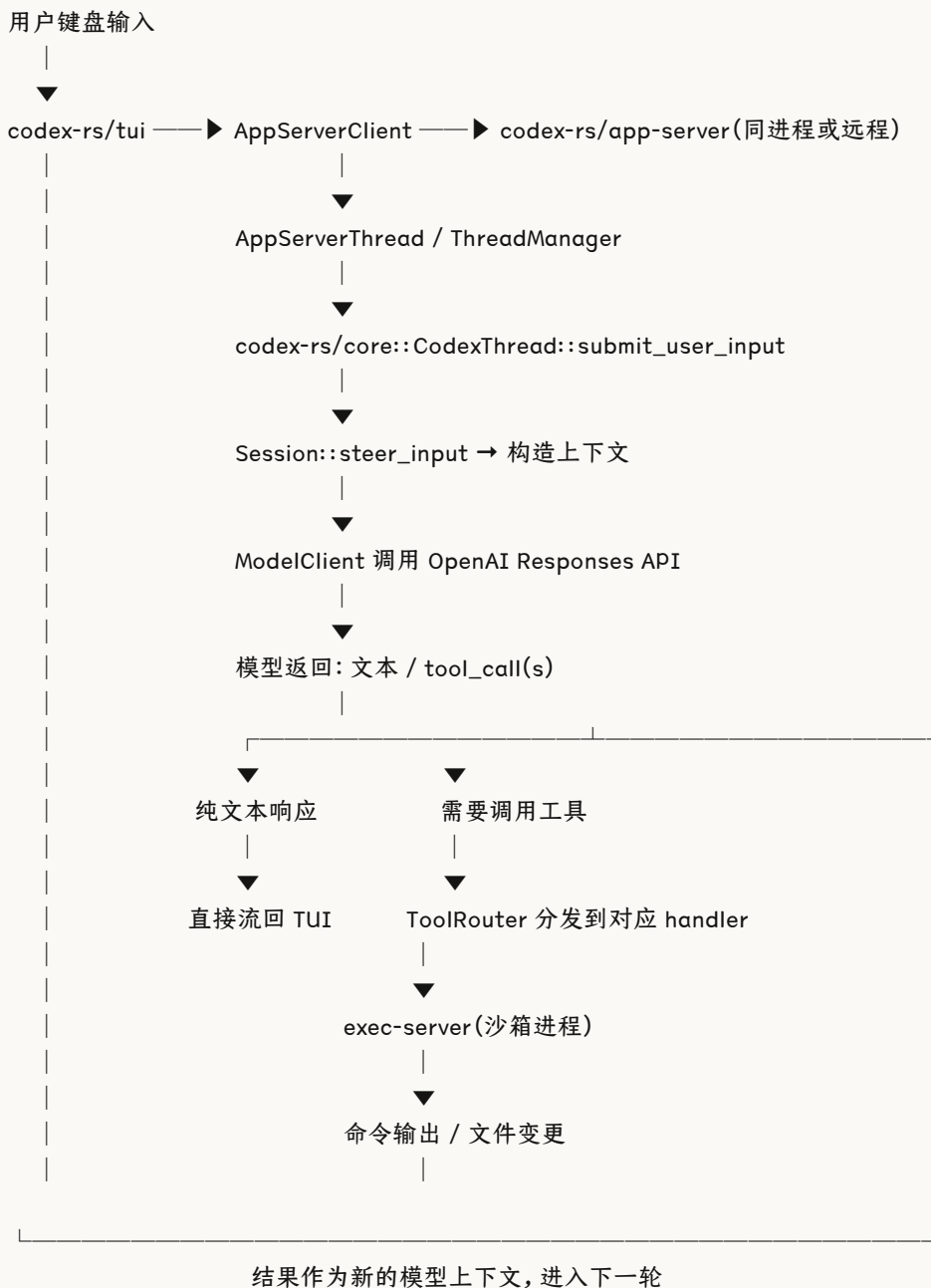
分层	代表 crate	主要职责
用户可见层	cli, tui, app-server-daemon	解析参数、启动终端 UI、启动桌面/远程服务入口
应用协议层	app-server-protocol, app-server-client	定义前后端通信协议，支持同进程/远程两种传输
核心编排层	core	Thread 生命周期、Session 状态机、模型调用、工具路由、Agent 控制
能力扩展层	skills, codex-mcp, hooks, ext/*, plugin	把具体能力(搜索、记忆、图片生成等)注册为工具或上下文片段，工具调用生命周期钩子
本地执行层	exec, exec-server	实际运行 shell 命令、管理沙箱、文件读写、进程生命周期
基础设施层	backend-client, config, state, rollout, history, login	模型后端、配置加载、会话持久化、日志/追踪、认证

核心数据流与关键抽象

一次交互请求从 TUI 经 app-server 协议进入 core 的 CodexThread，工具调用交给 exec-server 沙箱进程执行，结果回流为新的模型上下文形成多轮 ReAct 循环。Thread、Session、Turn、Agent 四个概念构成理解 Codex 的钥匙。

一次请求的完整路径

以用户在 TUI 里输入"帮我给这个项目加一个测试"为例：



四个关键观察

1. TUI 不直接调用模型，它通过 app-server 协议与后端交互。这让桌面应用、远程模式、JSONL 模式可以复用同一套后端。
2. CodexThread 是用户会话的句柄，真正的状态在 Session 里。一个 ThreadManager 可以管理多个 thread。
3. 工具调用走 exec-server，而不是在 core 里直接执行系统命令。这是安全隔离的关键。
4. 结果会回流到模型上下文，形成多轮 ReAct 循环，直到模型给出最终回答或达到预算上限。

Thread · Session · Turn · Agent

Thread(对话线程)

CodexThread 是用户与 Codex 一次连续对话的句柄，保存在 `codex-rs/core/src/codex_thread.rs`，内部持有 `Arc<Session>`(真正的状态机)、`SessionIo`(与 Session 通信的通道)、`session_configured`(会话配置快照)与 `rollout_path`(会话持久化路径)。它提供 `submit`、`steer_input`、`shutdown_and_wait` 等 API，是 UI 层与核心层交互的主要边界。

Session(会话状态机)

Session 在 `codex-rs/core/src/session/` 下，是核心循环的拥有者。它负责：维护消息历史与模型上下文；把用户输入追加到上下文中；调用模型；解析模型输出并分发工具调用；管理回合(turn)生命周期。

Turn(回合)

一次 Turn 通常对应"一次用户输入 → 模型响应 → 工具执行 → 结果返回"的完整循环。一个复杂请求可能包含多个内部 turn(例如模型先调用搜索工具，再调用编辑工具，最后调用测试工具)。

TurnContext 承载一个 turn 的瞬时状态：当前配置、模型信息、工具模式、环境选择等。

AgentControl(多 Agent 控制)

AgentControl 在 `codex-rs/core/src/agent/control.rs`，是 Codex 多 Agent 模式的控制中心，具体实现按职责拆在 `agent/control/` 子模块(`spawn.rs` 创建、`execution.rs` 并发与预算、`residency.rs` 驻留管理)。它维护一个 `AgentRegistry`(`agent/registry.rs`)，提供 `spawn_agent` 及其变体、`send_input / send_inter_agent_communication`(Agent 间通信)、`ensure_execution_capacity_for_turn_start`(并发与预算控制)。所有子 Agent 共享同一个 `session_id`，并通过 `Weak<ThreadManagerState>` 避免循环引用。

| 扩展、安全与设计取舍

Codex 的能力不是硬编码在 core 里的，而是通过 Extensions、MCP、Skills、Plugins、Hooks 五种机制扩展，注册进统一的 ExtensionRegistry；安全采用纵深防御加用户可控；一系列设计取舍支撑系统持续演进。

五种能力接入方式

机制	位置	用途	例子
Extensions	codex-rs/ext/*	原生 Rust 扩展，深度集成	guardian-v2、goal、image-generation、memories、web-search、skills、queue、agent
MCP	codex-rs/codex-mcp, codex-rs/mcp-server	通过 Model Context Protocol 接入外部工具	外部 MCP servers
Skills	codex-rs/skills, codex-rs/ext/skills	声明式提示词/工作流片段	内置技能模板 (skills crate 负责加载/调用, ext/skills 负责注册进扩展系统)
Plugins	codex-rs/core-plugins, codex-rs/plugin	连接器/插件系统	连接器、第三方插件
Hooks	codex-rs/hooks	工具调用生命周期钩子引擎	pre/post-tool 钩子、MCP 工具调用钩子 (2026-08 起 MCP 调用也接入 hooks 引擎)

统一注册中心

ExtensionRegistry(定义在 codex-rs/ext/extension-api/src/registry.rs)是统一注册中心。扩展可以注册：ToolContributor 提供新工具；ContextContributor 向模型上下文注入片段；TurnInputContributor / TurnItemContributor 在 turn 前后修改输入/输出；ThreadLifecycleContributor 监听 thread 启动/恢复/停止；ApprovalReviewContributor 参与审批决策。这种设计让新增能力时，大多数情况下不需要修改核心循环，只需要新增一个 crate 并注册到 ExtensionRegistryBuilder。

安全与权限模型

Codex CLI 的安全模型可以概括为纵深防御加用户可控：

- 1. 权限 Profile: 预定义的一组文件/网络/执行策略，例如 read-only、workspace、dangerous。

- 2. 沙箱模式(SandboxMode): none、seatbelt、bwrap、windows 等, 决定命令在什么隔离环境中运行。
- 3. 审批策略(AskForApproval): on-request(默认, 模型自行决定何时请求审批)、granular(按类别开关: 沙箱审批、execpolicy 规则、技能脚本、request_permissions、MCP elicitation)、never。always 与 untrusted 已移除(2026-08, #39630)。
- 4. exec-server 隔离: 实际执行命令的进程与核心进程分离, 支持远程执行和最小权限文件系统视图。
- 5. 网络代理与审计: network-proxy 对出站网络进行控制和日志记录。

这些配置都通过 Config 分层加载, 并可以在 AGENTS.md 项目级文档中进一步约束。

设计取舍

取舍	选择	替代方案	原因
单一大仓库 vs. 多仓库	单一大仓库(monorepo)	各 crate 独立仓库	协议变更容易同步, Bazel/Cargo 可以统一构建
core 持续膨胀 vs. 严格拆 crate	鼓励拆 crate, 抵抗向 core 加代码	所有功能放 core	避免 core 变成无法维护的巨型 crate
工具执行在 core 内 vs. exec-server	exec-server 隔离	直接执行系统命令	安全、可审计、可远程
UI 与后端同进程 vs. 协议分离	app-server 协议分离	UI 直接调 core API	支持 TUI、桌面、远程、JSONL 多种前端
配置集中 vs. 分层	分层配置 + lockfile	单一全局配置	支持用户/项目/CLI 覆盖, 且可锁定
多 Agent 共享 registry vs. 完全独立	同 session 下共享 AgentControl	每个 Agent 独立进程	便于预算、并发、通信统一管控

小结

Codex CLI 的架构可以总结为: 一个以 core 为中心的本地 AI 代理操作系统。core 负责把用户输入、上下文、模型、工具编织成持续的 Turn 循环; tui / app-server 负责把这套能力包装成不同交互形态; exec-server 负责在受控环境中执行模型产生的动作; ext / mcp / skills 负责把不断新增的能力接入系统而不污染核心。

架构主线

把模型交互与本地执行解耦, 把核心循环与具体能力解耦 —— 这是 Codex CLI 能持续演进而不变成巨型 crate 的根本原因。

附录：关键实现入口

本章提到的所有概念都有明确的文件级入口，构成后续深入阅读的导航表。

A. 关键实现入口

概念	主要文件	说明
CLI 入口	codex-rs/cli/src/main.rs	命令解析、子命令分发
NPM 包装	codex-cli/bin/codex.js	按平台找到 Rust 二进制并 spawn
TUI 入口	codex-rs/tui/src/lib.rs	run_main、run_ratatui_app
核心库	codex-rs/core/src/lib.rs	codex-core 的公开 API
Thread	codex-rs/core/src/ codex_thread.rs	CodexThread
Agent 控制	codex-rs/core/src/agent/ control.rs	AgentControl、AgentRegistry
工具路由	codex-rs/core/src/tools/ router.rs	ToolRouter
工具注册	codex-rs/core/src/tools/ registry.rs	工具元数据注册
执行入口	codex-rs/exec/src/lib.rs	非交互式 codex exec
执行服务器	codex-rs/exec-server/src/lib.rs	沙箱执行服务
扩展 API	codex-rs/ext/extension-api/src/ lib.rs	扩展能力 trait
扩展注册表	codex-rs/ext/extension-api/src/ registry.rs	ExtensionRegistry
配置加载	codex-rs/core/src/config/mod.rs	Config 构建与约束
协议类型	codex-rs/protocol/src/	跨 crate 共享的事件、模型、配置类型
会话历史类型	codex-rs/history/src/lib.rs	模型历史与 rollout 持久化领域类型(2026-08 从 core 拆出, #37871)

B. 参考资料

- OpenAI Codex CLI 官方仓库与文档: <https://github.com/openai/codex>

- [how-codex-works 系列 · 第 1 章: 01-overview.md](#) (本文来源)

C. 下一章预告

下一章(启动与命令分发)会沿着 `codex-cli/bin/codex.js` → `codex-rs/cli/src/main.rs` → `run_interactive_tui` / `codex_exec::run_main` 这条路径, 解释 Codex CLI 是如何从一次 shell 调用启动起来的。